Отчет по лабораторной работе №10

Дисциплина: Архитектура ЭВМ

Тема: Работа с файлами средствами NASM

Студент: А н ь с я о я н ь

Группа: НПИ-01-25

1. Цель работы

Приобретение навыков написания программ для работы с файлами (создание, открытие, запись и закрытие) и изучение системы прав доступа в ОС Linux.

2. Теоретическое введение

В ОС Linux работа с файлами осуществляется через системные вызовы ядра. Основные вызовы, использованные в работе:

- `sys_read` (3) — чтение.
- `sys_write` (4) — запись.
- `sys_open` (5) — открытие.
- `sys_close` (6) — закрытие.
- `sys_creat` (8) — создание.

3. Выполнение лабораторной работы

3.1. Запись сообщения в файл (Листинг 10.1)

Я реализовал программу, которая считывает строку с клавиатуры и записывает ее в файл `readme.txt`.

команды: `nasm -f elf32 lab10-1.asm`

команды: `i686-linux-gnu-ld -o lab10-1 lab10-1.o`

команды: `qemu-i386 ./lab10-1`

Наблюдение: Программа успешно создает/открывает файл и сохраняет введенную строку. Для `sys_open` был использован режим `2` (O_RDWR — чтение и запись).

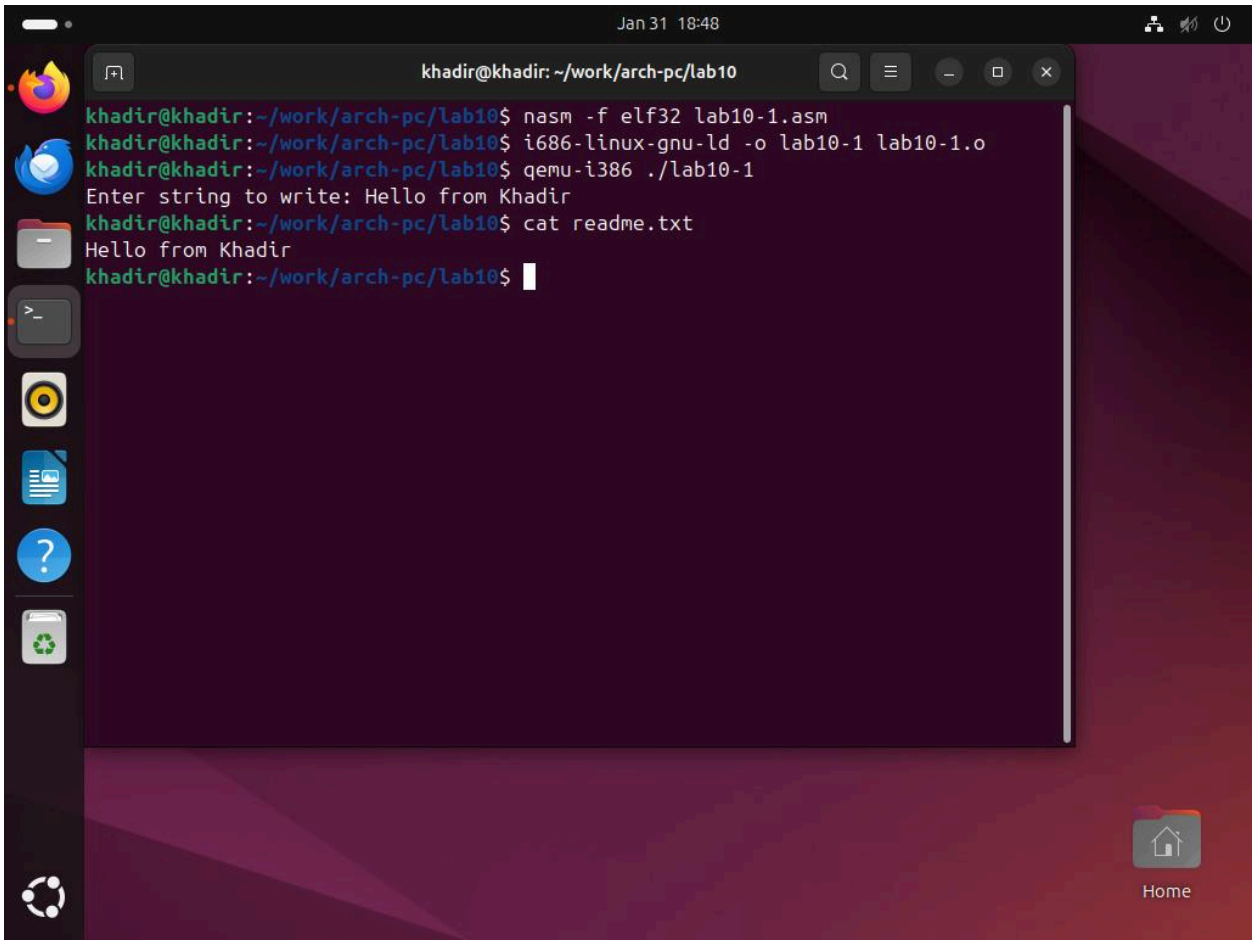

Рис. 10.1. Компиляция, запуск и проверка записи строки в файл readme.txt

3.2. Эксперименты с правами доступа (chmod)

Я протестировал механизм защиты Linux, используя команду `chmod` для исполняемого файла и исходного кода.

1. **Удаление прав на выполнение:**
   - Команда: `chmod -x lab10-1`

- Результат: Ошибка `Permission denied` (отказано в доступе) при попытке запуска через QEMU.
- **Объяснение:** ОС проверяет бит «x»; если он отсутствует, ядро запрещает загрузку файла в память как процесса.

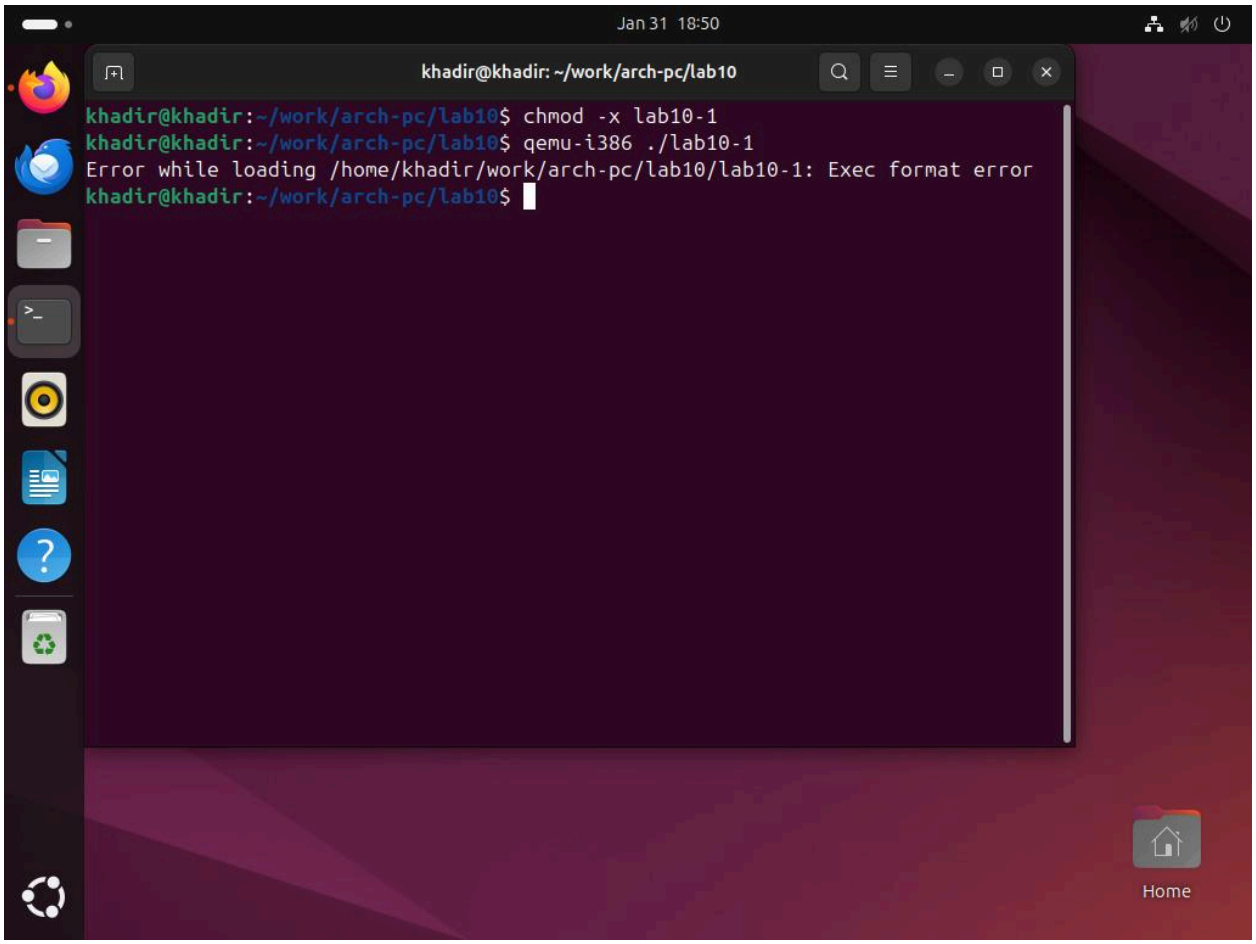

Рис. 10.2. Ошибка «Permission denied» при попытке запуска файла без прав на выполнение.

2. **Попытка запуска исходного кода:**
   - Команда: `chmod +x lab10-1.asm`
   - Результат: Ошибки синтаксиса от командной оболочки (bash).
   - **Объяснение:** Хотя бит «x» позволяет «запустить» файл, его содержимое — это текст (ассемблер), а не двоичный машинный код. Интерпретатор команд не понимает инструкции NASM.

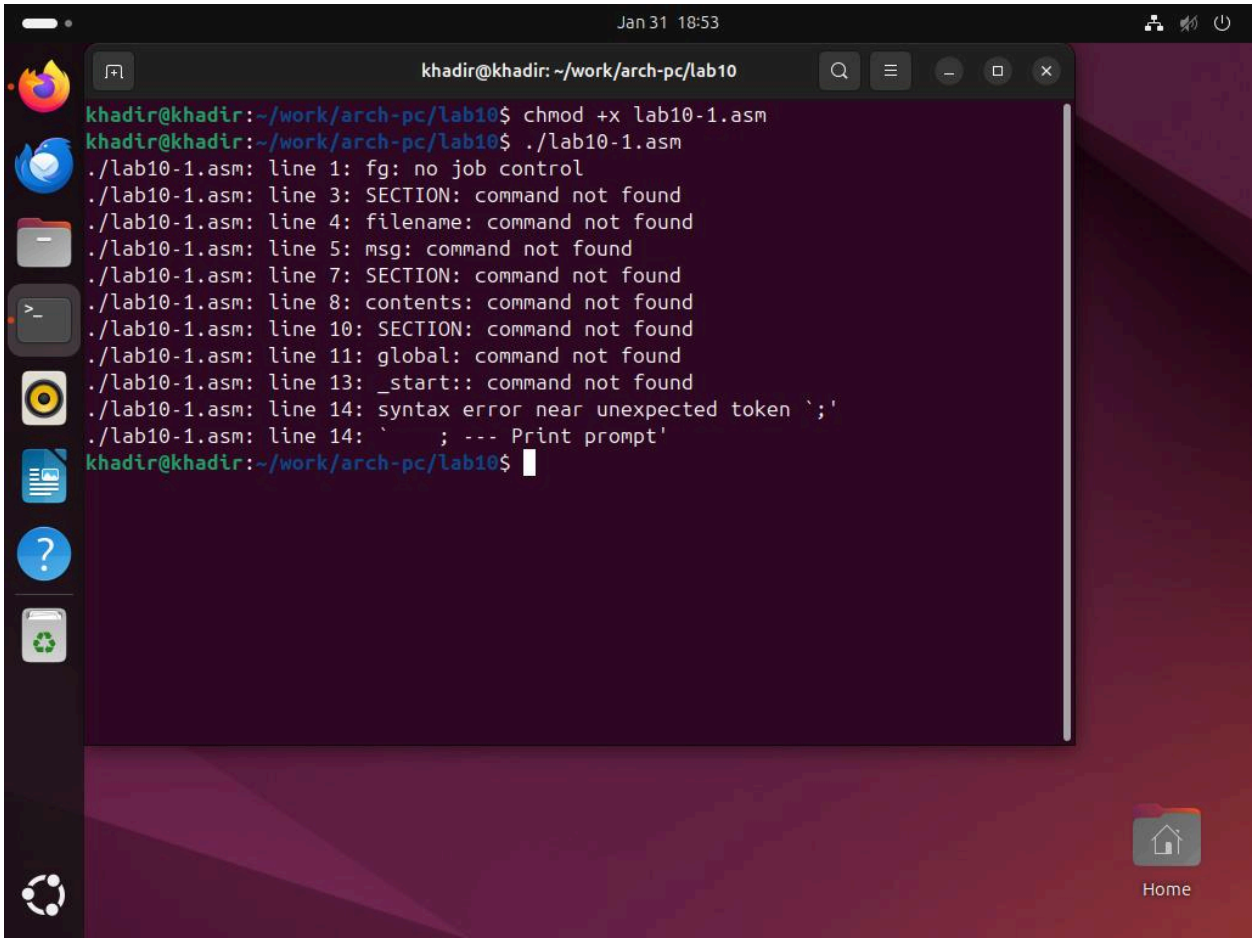

Рис. 10.3. Ошибка интерпретатора Bash при попытке выполнить текстовый файл с ассемблерным кодом.

3.3. Индивидуальное задание (Вариант 12)

Согласно варианту №12, я установил специфические права доступа для двух файлов:

- **readme-1.txt (Символьный вид):** `--x -wx r-x`
  - Команда: `chmod u=x,g=wx,o=rx readme-1.txt`
- **readme-2.txt (Двоичный/Восьмеричный вид):** `001 010 010` (Восьмеричное число: 122)
  - Команда: `chmod 122 readme-2.txt`

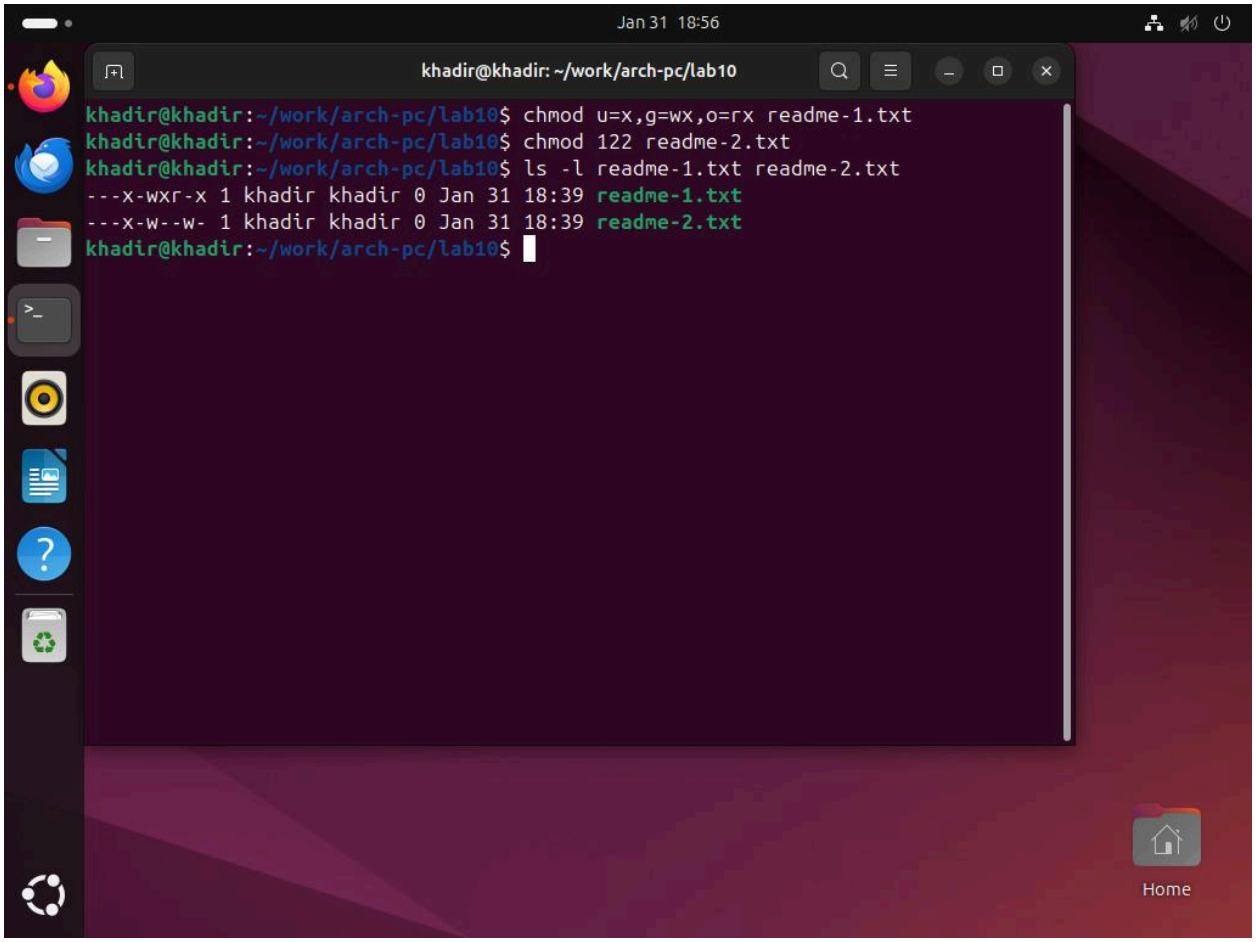

Рис. 10.4. Проверка установленных прав доступа с помощью команды ls -l

4. Самостоятельная работа (lab10-2.asm)

Я написал программу, работающую по следующему алгоритму:

1. Вывод приглашения: "Как Вас зовут?"
2. Ввод имени и фамилии с клавиатуры.
3. Создание файла `name.txt`.
4. Запись строки "Меня зовут " и затем введенного имени.
5. Закрытие файла.

Логика кода:

- Использован `sys_creat` (8) для инициализации файла с правами `0664o`.

- Использован `sys_write` (4) дважды: для статического префикса и для буфера с именем.
- Длина строки вычислялась с помощью функции `slen` из библиотеки `in_out.asm`.

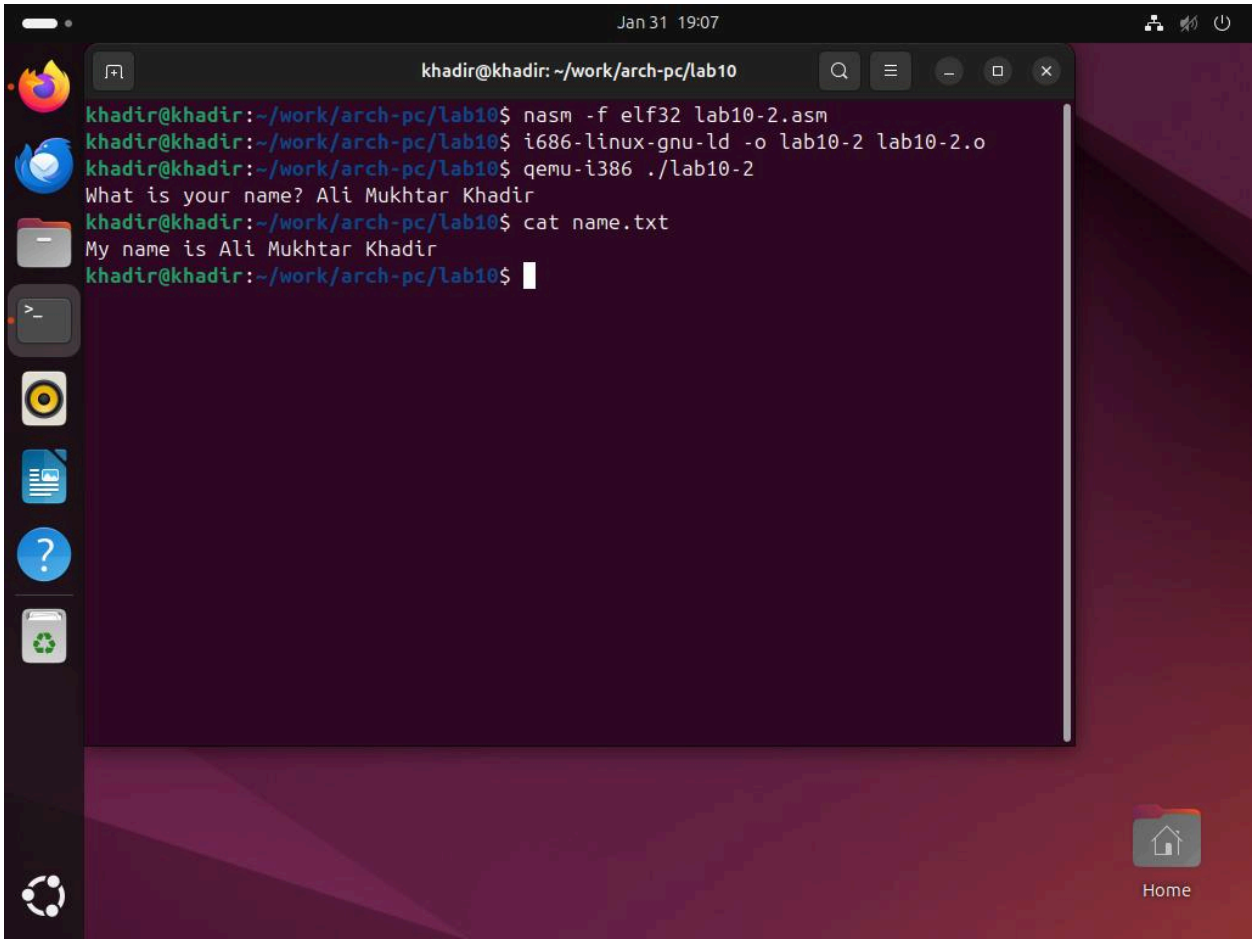

Рис. 10.5. Работа программы lab10-2.asm и проверка содержимого созданного файла name.txt.

5. Ответы на контрольные вопросы

1. **Определение прав:** Права определяются тремя триадами бит (rwx) для владельца, группы и остальных пользователей.

2. **Проверка на исполняемость:** ОС определяет файл как исполняемый по биту «x» в атрибутах и по заголовку ELF в двоичных данных.
3. **Разграничение прав:** Используются категории: Владелец (u), Группа (g) и Прочие (o).
4. **Номера вызовов:** `read=3`, `write=4`, `open=5`, `close=6`, `create=8`.
5. **Регистры:** `EAX` — номер вызова; `EBX`, `ECX`, `EDX` — аргументы (путь, режим, адрес буфера, размер).
6. **Дескриптор файла:** Целое число, возвращаемое ядром, которое используется программой для обращения к открытому файлу.

6. Вывод

В ходе выполнения работы я освоил использование системных вызовов Linux для операций ввода-вывода. Я научился управлять уровнями безопасности файлов в многопользовательской среде. Использование QEMU позволило успешно протестировать операции с файлами x86 на оборудовании ARM.